T13 IntelliJ IDEA Intelligent Teaching Assistant Plugin

1. Team Member Information

Name	Student ID	Contact Number	Email Address
程瑜	2350280	18918512449	c2483166226@163.com
王小萌	2351882	19117171587	2351882@tongji.edu.cn
程琮越	2351759	15304863669	2351759@tongji.edu.cn
路清怡	2352996	18936966136	849014041@qq.com

2. Complete List of Implemented Features and Capabilities

This plugin revolves around the teaching assistant scenario, implementing two core capabilities: F0 (Course Q&A) and F2 (AI-driven Code Modification).

-F0: Basic Functionality – Course Teaching Assistant (TA) Chatbot

F0.1: Course Q&A (Tool Window)

Provides a dedicated tool window on the right side of the IDE for course Q&A and message display. Supports general question input, loading status, and error prompts. Messages are rendered chronologically for a clear and stable experience. In addition to the basic course material library, users can upload PDF materials in the tool window; the system automatically indexes them in the background and visually provides feedback on indexing progress and results.

F0.2: Editor Context Awareness (Right-Click Menu)

Users can select code in the editor and trigger “ask selected code to AI Chat” via right-click, automatically capturing the file name and line range, and adding the selected code as context to the chat window. Notifications confirm successful context capture, facilitating targeted Q&A and analysis around specific code.

F0.3: Retrieval-Augmented Generation (RAG) and Source Citation

Documents are parsed page by page and indexed using a hybrid approach of

keywords and vectors. During retrieval, results from both types are weighted and fused to improve relevance and robustness. Answers are strictly constructed based on the retrieved context, with sources cited at the end in the format “file name + page range” for academic verification. Supports topK settings and sorting of retrieval results; citations are de-duplicated at the UI layer for clarity and conciseness.

F0.4: Model Switching and Selection

A dropdown menu for model selection is provided at the bottom of the Q&A tool window, supporting instant switching. The service layer maintains the current model state and provides a unified interface, coordinating with the retrieval process. Switching affects only subsequent answers and source citations, without retroactively altering already displayed message history.

-F2: Extended Functionality – AI-Driven Source Code Modification

F2.1: AI Code Modification Tasks (Instruction-Driven)

The editor’s right-click menu provides a “Modify with AI” entry; users can describe their goals in natural language (e.g., refactoring, bug fixing, adding comments). The backend generates candidate modifications in a streaming manner, incrementally displayed in the right-side panel. The initial state shows “Thinking, please wait…”

F2.2: Multi-Turn Dialogue and Context Iteration

Context (original code, instructions, and model state) is preserved within the same session, supporting iterative instructions. Users can fine-tune or retry based on the currently generated code in the right panel. Sessions are cleared upon closing the dialog or completing the application, preventing contamination of other files’ editing context.

F2.3: Professional Diff Review Experience

Uses the IDE’s native Diff viewer for side-by-side review: the left shows the original code, the right shows AI suggestions (editable). Differences are highlighted, and AI output can be adjusted immediately. Generation progress and status are clearly displayed in the dialog to prevent misuse.

F2.4: Safe Editing, Application, and Rejection Mechanism

Before applying changes, users can directly edit AI-generated code in the right suggestion panel and fine-tune it. Only after clicking “Apply Modification” will the

system safely replace the selected content; by default, source files are not modified directly. Users may also choose “Reject” or close the window to discard changes; errors are clearly prompted without affecting existing code.

3. Quick User Guide

3.1 Feature Overview

TA-Assistant is an IntelliJ IDEA plugin that implements the F0 basic function (RAG Q&A system) and F2 extended function (AI code modification) required by the assignment. Key features include:

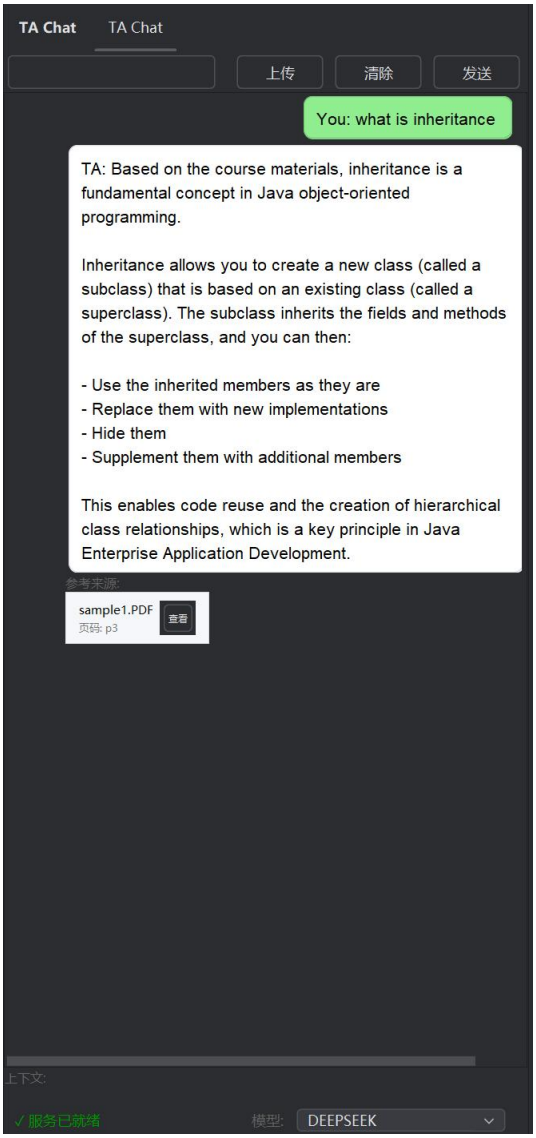
- Intelligent Q&A: Retrieval-Augmented Generation (RAG) based on course materials
- Source Citation: Automatically label the document and page number of the answer source
- Context Awareness: Ask questions with selected code
- AI Code Modification: Preview and review changes using a Diff viewer
- Multi-format Document Management: Support for PDF/Word/PowerPoint
- Multi-model Support: Three AI models (DeepSeek/Qwen/GLM)

3.2 F0: Intelligent Q&A System

3.2.1 Open the TA Chat Window

Click the "TA Chat" icon in the toolbar on the right side of the IDE.

The interface layout includes a top operation bar (upload, clear, send), user messages (green bubbles), AI answers (white bubbles), citation cards (click to jump directly to the original document), a code context area (for asking questions with selected code), and a bottom status bar (server status and model selection).



3.2.2 Regular Question Asking

Operation steps:

1. Enter a question in the input box (e.g., "What is inheritance")
2. Click the "Send" button
3. The system executes the RAG process:
    - Retrieve relevant document chunks from the knowledge base
    - Construct a Prompt containing background knowledge
    - Call the LLM to generate an answer
4. The answer is displayed in the chat window with source citation cards attached

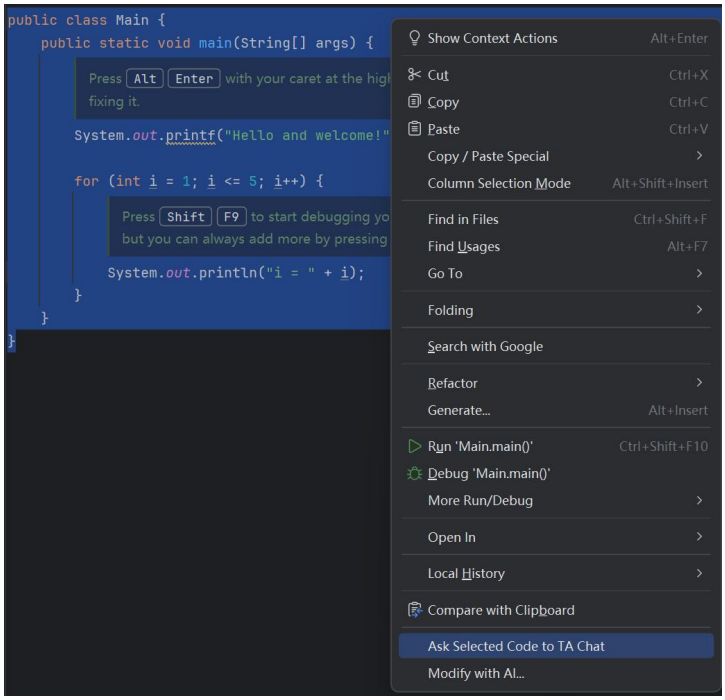
3.2.3 Source Citation Function (F0 Core Requirement)

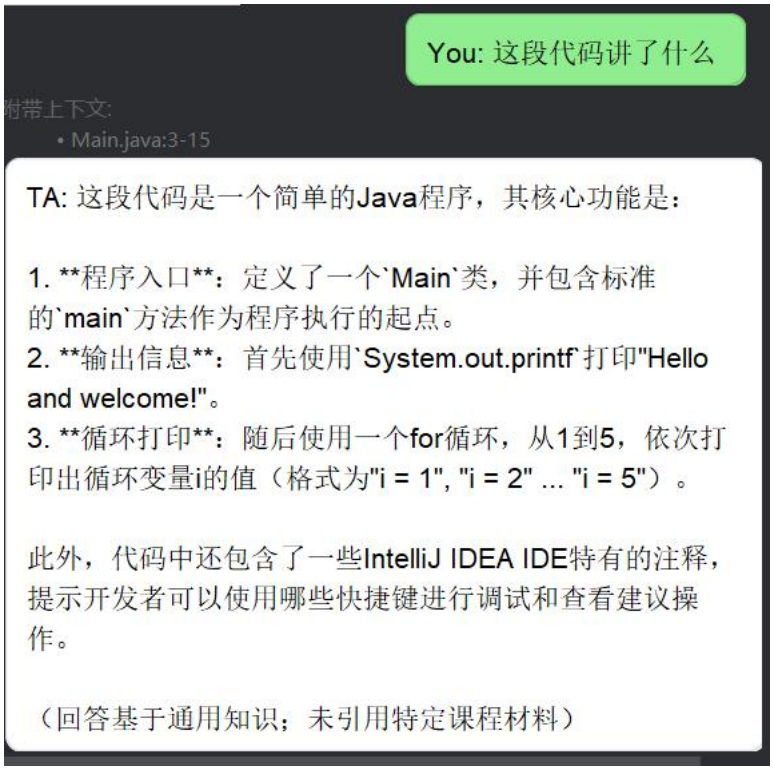
The system strictly follows the citation rules required by the assignment:

- Scenario 1: Answers based on course materials
    - The LLM inserts [cite: File Name, Page Range] tags in the answer
    - The system parses the tags and generates citation cards
    - Each card displays the source file name, page range, and a "View" button (click to open the original document)
- Scenario 2: Answers based on general knowledge
    - When there is no relevant data in the knowledge base, the LLM returns a [GENERAL_KNOWLEDGE] tag
    - The system automatically appends a disclaimer: "(Answer based on general knowledge; no specific course materials cited)"
    - No citation cards are displayed

3.2.4 Context Awareness Function (F0 Core Requirement)

- Right-click menu integration:
    1. Select a code snippet in the editor
    2. Right-click → Choose "Ask TA About Selected Code"





3. The TA window is activated automatically, and the code is temporarily stored as context 4. The system displays a notification: "Selected code has been temporarily stored as context" 5. Enter a question in the chat window, and the system will generate an answer combining the code context

- Context display:
  - The temporarily stored code is displayed in the "Context" area at the bottom of the chat window
  - Format: File Name:Start Line-End Line (e.g., Example.java:12-25, AskSelectedCodeAction.java:44-51)
  - Automatically cleared after sending the question
  - Multiple contexts can be added, and the system will aggregate all contexts

3.2.5 AI Model Switching

Switch AI models at the bottom of the chat window:

Model	Model Name	Application Scenarios

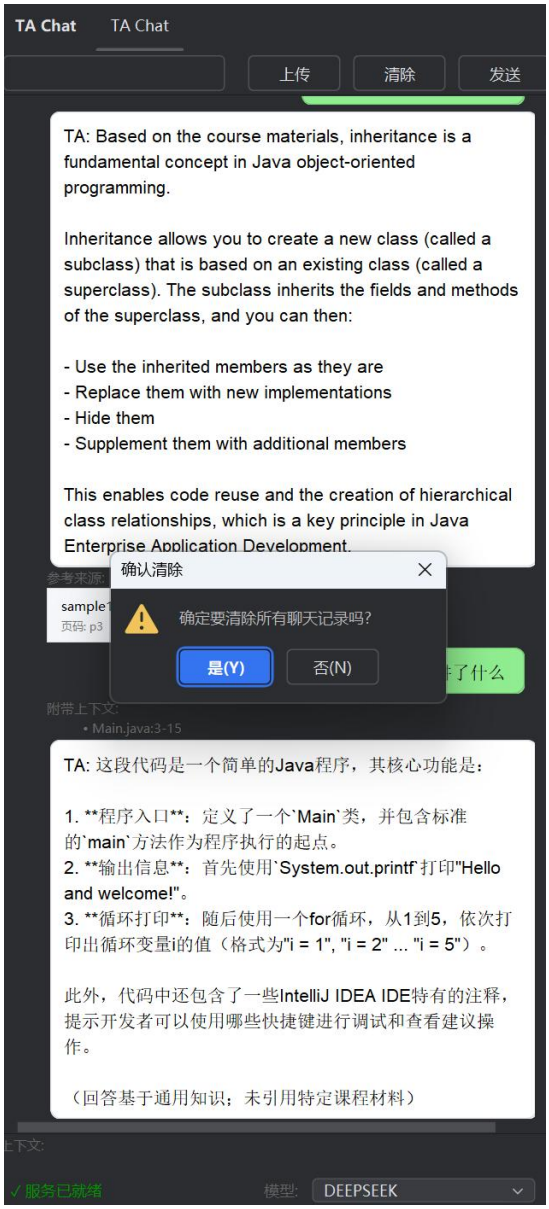
Model	Model Name	Application Scenarios
DeepSeek	deepseek-ai/DeepSeek-V3.2-Exp	General Q&A, good overall performance
Qwen	Qwen/Qwen3-VL-32B-Thinking	Balanced performance, strong reasoning ability
GLM	zai-org/GLM-4.5	Code-related tasks, programming-focused

After switching models, the system will immediately apply the new model to process subsequent questions.

3.2.6 Clear Chat History

Click the "Clear" button in the chat window:

1. The system pops up a confirmation dialog box
2. Clear all messages and contexts after confirmation
3. Chat history will be completely deleted



3.3 Document Management

3.3.1 Supported Document Formats

The system supports three course material formats:

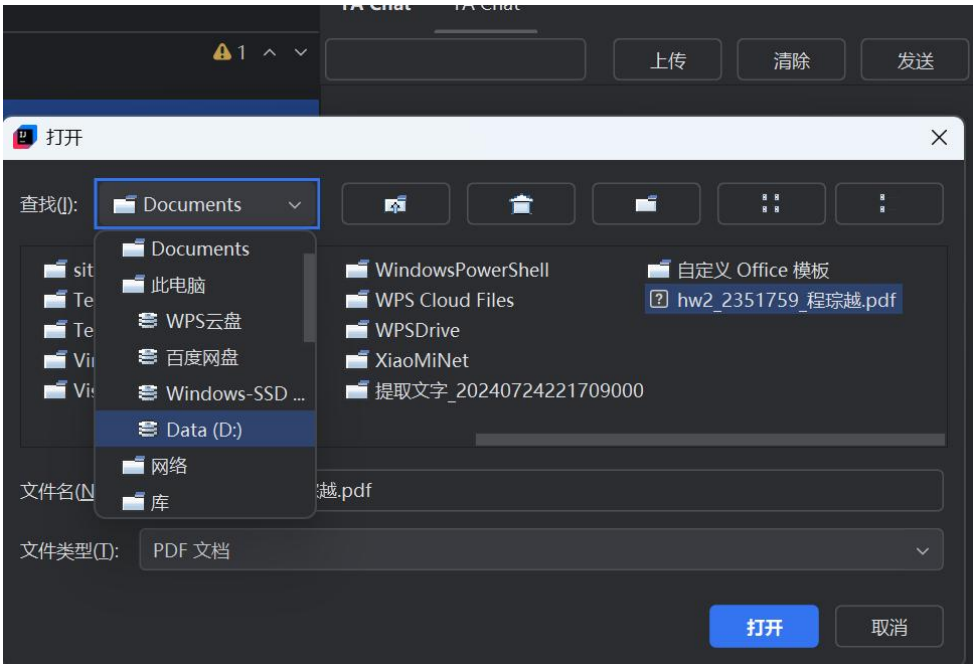
Format	Extension	Parsing Strategy	Application Scenarios
PDF	.pdf	Parse by page	Lecture slides, textbooks
Word	.docx	Sliding window chunking (800 characters/chunk, 100-character overlap)	Assignment instructions, technical documents

Format	Extension	Parsing Strategy	Application Scenarios
PowerPoint	.pptx	Parse by slide	Course PPT

3.3.2 Upload Documents

Operation steps:

1. Click the "Upload" button in the chat window
2. Select a document in the file picker



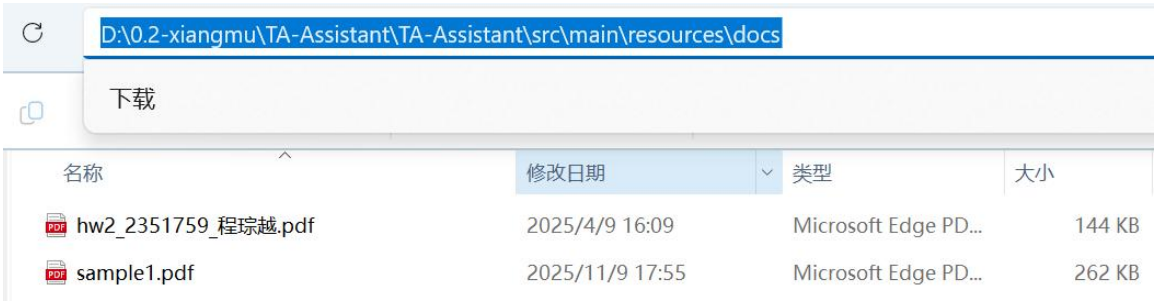
3. The system displays a "Indexing document..." prompt 4. Wait for the indexing to complete

Indexing process:

- Lucene keyword indexing: Establish a full-text search index to support exact matching
- API vector indexing: Establish a semantic search index to support similarity retrieval
- After completion, display "Document saved to docs directory and indexed successfully"



- The saved document can be seen in the docs directory



3.3.3 Document Preprocessing Details

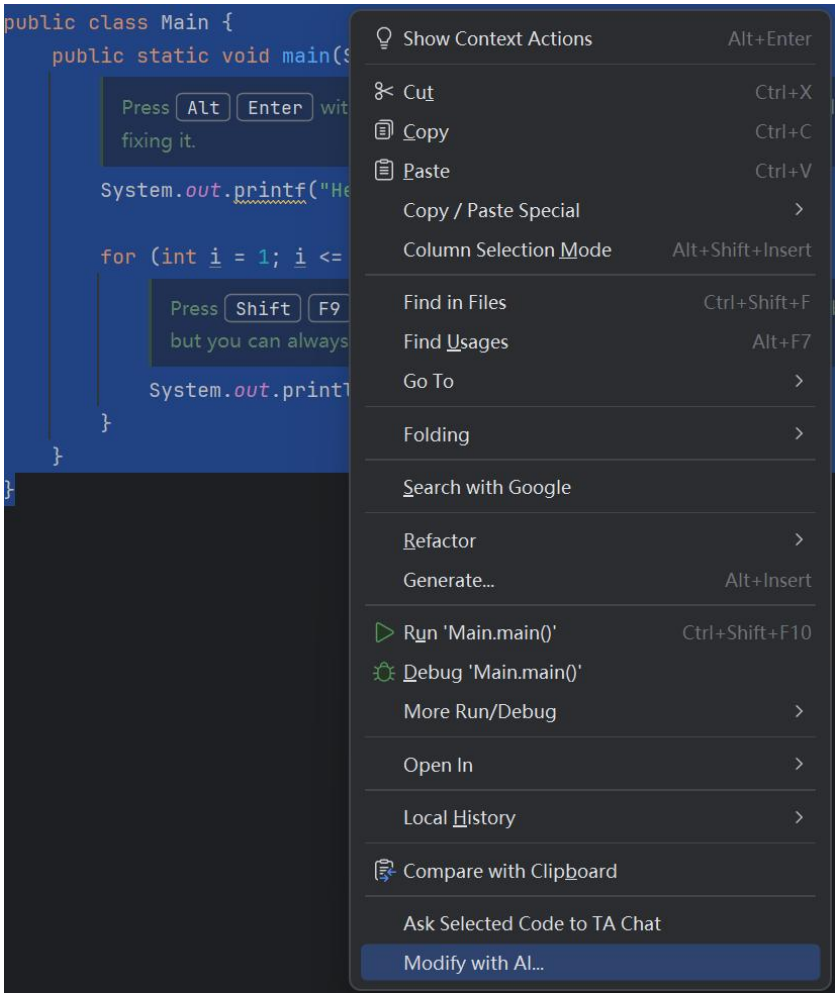
- PDF parsing: Extract text by page, each page as an independent knowledge fragment, retain page number information for citation
- Word parsing: Use a sliding window strategy, each chunk has a maximum of 800 characters, adjacent chunks overlap by 100 characters, split at sentence boundaries to maintain semantic integrity, suitable for long continuous text
- PowerPoint parsing: Extract text by slide, each slide as a knowledge fragment, retain slide number for citation

3.4 F2: AI Code Modification

3.4.1 Trigger Code Modification

Operation steps:

1. Select the code to be modified in the editor
2. Right-click → Choose "Modify Code with AI"



3. In the pop-up window: - Select an AI model (DeepSeek/Qwen/GLM) - Enter modification instructions (e.g., "Add logs", "Refactor with Singleton pattern", "Fix null pointer exception", ModifyCodeWithAIAction.java:90-91)



4. Click "Send"

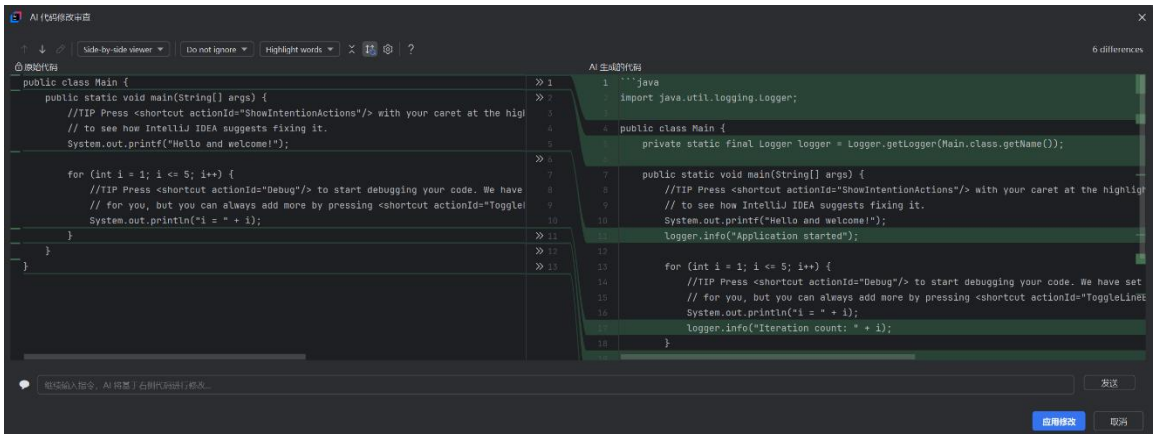
3.4.2 Diff Viewer (F2 Core Requirement)

The system uses IntelliJ's built-in Diff framework to display proposed changes.

The interface layout includes the original code (read-only) on the left, the AI-generated code (editable) on the right, supports multi-turn conversations, and has two options: Cancel and Apply Modifications.

AI code is generated in real-time streaming:

- AI code is displayed in chunks in the right panel in real-time
- Users can see the code generation process as it happens
- Automatically cleans up Markdown code block tags



3.4.3 User Review and Confirmation (F2 Core Requirement)

Review options:

- Direct editing: Manually modify the AI-generated code in the right panel
- Multi-turn conversation: Continue to put forward modification requirements in the input box at the bottom
    - Example: "Add exception handling"
    - The AI continues to modify based on the current code in the right panel
    - Multiple iterations are allowed until satisfied
- Apply modifications: Click the "Apply Modifications" button to write the code to the editor
- Cancel: Click "Cancel" to discard all changes

3.4.4 Supported Modification Tasks

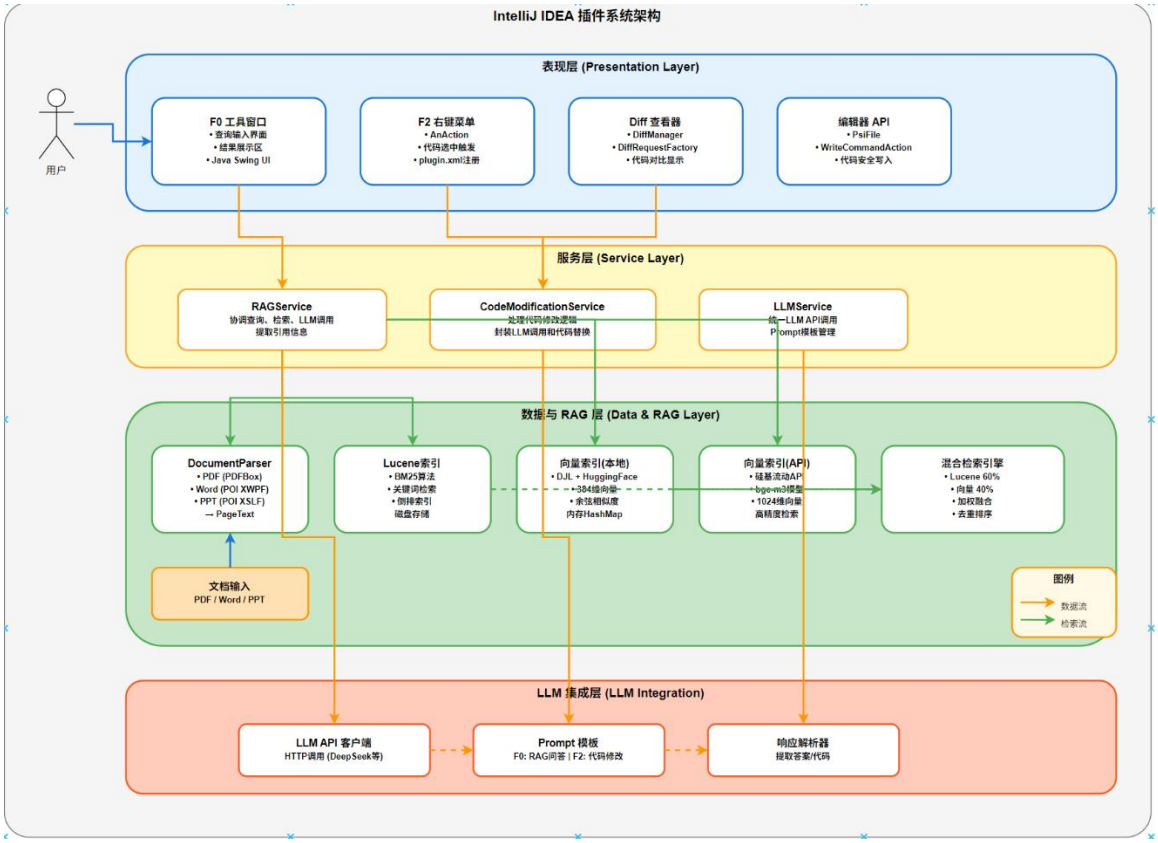
According to the assignment requirements, the system supports:

- Design pattern refactoring (e.g., Singleton pattern, Factory pattern)
- Bug fixing
- Code optimization (performance, readability)
- Adding logs/comments
- Exception handling

4. System Architecture and Technical Implementation

4.1 System Architecture and Components

Our plugin adopts a layered architecture design to ensure decoupling and maintainability of F0 (RAG Q&A) and F2 (AI Code Diff) functionalities.



4.1.1 Presentation Layer

The presentation layer uses Java Swing to build the F0 Tool Window UI, and registers F0 and F2 context menus through AnAction classes and plugin.xml. F0 provides an independent query input interface and result display area, while F2 is triggered from the

context menu when code is selected in the editor. The Diff display functionality uses IntelliJ Platform API (DiffManager, DiffRequestFactory) to invoke F2's built-in Diff viewer, enabling visual comparison before and after code modifications.

4.1.2 Service Layer

The service layer contains three core service components:

- **RAGService**: Coordinates the complete RAG workflow for F0, manages the lifecycle of user queries, document retrieval and LLM invocation, and handles post-processing of retrieval results and extraction of reference information.
- **CodeModificationService**: Handles the code modification logic for F2, encapsulating LLM invocation and safe code replacement operations.
- **LLMService**: Provides unified LLM API invocation encapsulation and supports Prompt template management for different scenarios.

4.1.3 Data & RAG Layer

The Data & RAG layer is responsible for document processing and retrieval functionality, consisting of four subsystems:

Document Parsing Subsystem

DocumentParser is a multi-format document parser that supports three formats: PDF (using Apache PDFBox for page-by-page text extraction), Word (using Apache POI XWPF for paragraph text extraction), and PowerPoint (using Apache POI XSLF for slide text content extraction), with unified output in PageText { fileName, pageNumber, text } format.

Keyword Indexing Subsystem

LuceneKnowledgeStore implements full-text retrieval based on Apache Lucene, uses StandardAnalyzer for Chinese and English word segmentation, supports keyword matching with BM25 similarity algorithm, and stores each document chunk with content, source, and page fields.

Vector Indexing Subsystem

The system integrates cloud-based embedding API (ApiEmbeddingIndex), calling SiliconFlow API's BAAI/bge-m3 model to generate 1024-dimensional high-quality vectors, suitable for production environments requiring higher retrieval accuracy.

Hybrid Retrieval Engine

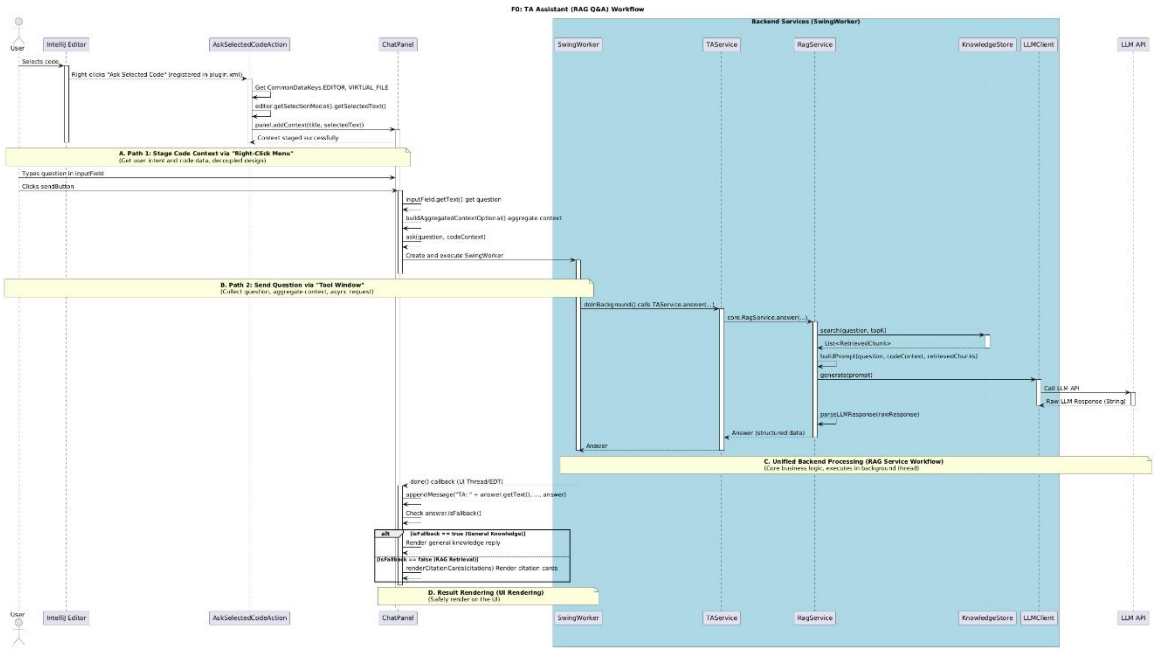
HybridKnowledgeStore fuses keyword and vector retrieval strategies, adopting a retrieval weight configuration of Lucene 60% + Vector 40% (adjustable via alpha parameter), using weighted score fusion algorithm fusedScore = α × luceneScore + (1-α) × vectorScore, and implementing deduplication mechanism based on the unique key fileName:pageNumber.

4.1.4 LLM Integration Layer

The LLM integration layer invokes large model APIs (such as DeepSeek) through HTTP clients, with specialized Prompt engineering designed for different scenarios: F0 scenarios use RAG Prompts optimized for course knowledge Q&A, and F2 scenarios use structured Prompts for code modification. The response parsing module handles LLM output, extracting pure code or formatted answers.

4.2 Overall Workflow

4.2.1 F0: Course Teaching Assistant (RAG Q&A) Workflow



F0 Workflow Diagram

The F0 (Course TA) function is the core of the plugin. Its workflow connects the frontend UI, the IntelliJ framework, the backend service, and the LLM API. The design of this process focuses on responsiveness (a non-blocking UI) and modularity (separation of concerns).

The flow is divided into two main starting points (A and B), which eventually converge into a unified backend process (C) and return to D.

A. Path 1: Staging Code Context via "Right-Click Menu"

This path is responsible for obtaining user intent and code data from the editor.

1. **Trigger:** The user selects code in the IntelliJ editor and right-clicks, triggering the actions.AskSelectedCodeAction class registered in plugin.xml. The AskSelectedCodeAction.actionPerformed() method is called. It retrieves CommonDataKeys.EDITOR (the editor) and CommonDataKeys.VIRTUAL_FILE (the virtual file) from the AnActionEvent data and calls editor.getSelectionModel().getSelectedText() to extract the selected code String.
2. **UI Staging:**AskSelectedCodeAction finds the ChatPanel instance via ToolWindowManager. It calls the panel.addContext(title, selectedText) method. The context (ContextItem) is staged in a List within the ChatPanel.
   This decoupled design allows "context acquisition" and "question sending" to occur asynchronously, preparing for Path B.

B. Path 2: Sending the Question via "Tool Window" This path is responsible for collecting the user's question, aggregating context, and **asynchronously** initiating the backend request.

1. **Trigger:** The user types a question into the inputField (a JTextField) in the ChatPanel and clicks the sendButton (a JButton).
2. **Context Aggregation:** The sendButton's ActionListener is triggered. It gets the question by calling inputField.getText() and calls the buildAggregatedContextOptional() method to aggregate the List<ContextItem> (staged in Path A) into an Optional<String> codeContext.
3. **Async UI Handling:** ChatPanel.ask(question, codeContext) is called. This method creates a SwingWorker<Answer, Void> instance and calls service.answer(...)

within its doInBackground() method. Since service.answer() involves RAG retrieval and an LLM network call, it is a time-consuming operation. If called directly on the UI thread (EDT), it **would cause the entire IntelliJ interface to freeze**. SwingWorker moves the time-consuming work (Path C) to a background thread, ensuring the UI remains responsive.

C. Unified Backend Processing (RAG Service Workflow) This path is the core business logic (handled by Group B) and is executed in the SwingWorker's background thread.

1. **Service Dispatch:** SwingWorker calls TAService.answer(), which in turn calls core.RagService.answer(...). RagService encapsulates the complexity of the RAG process, hiding the details of "retrieval," "prompt building," and "LLM calls" from the upper layer (UI).

2. **RAG Retrieval:** RagService calls knowledgeStore.search(question, topK). This call is delegated to the KnowledgeStore interface (e.g., HybridKnowledgeStore), making the RAG retrieval dependent on the KnowledgeStore abstraction, achieving code decoupling between different functional modules.

3. **Prompt Building:** RagService calls the private buildPrompt(...) method.

- **Technical Implementation:** It assembles the "user question," "code context," and the List<RetrievedChunk> returned from Group A into a complex String Prompt (containing RAG rules) that meets the LLM's requirements.

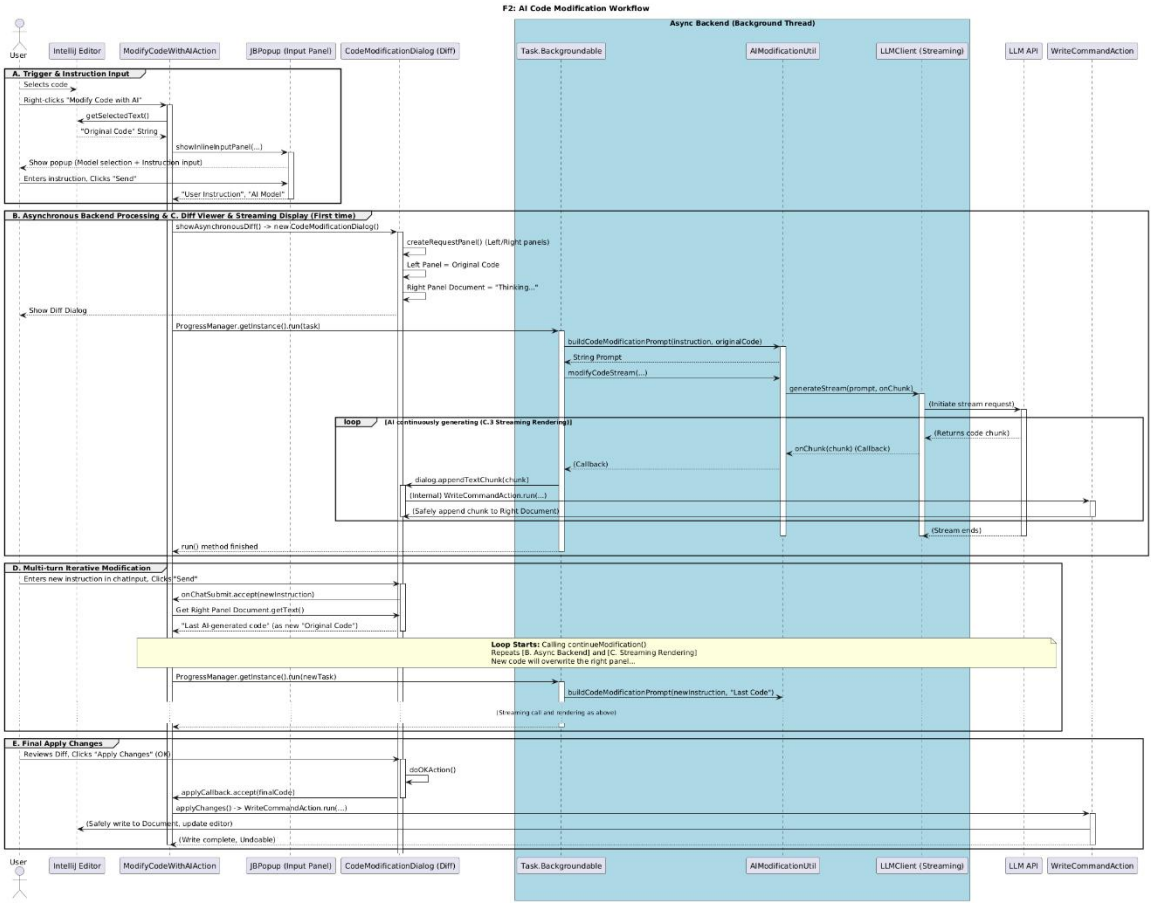
4. **LLM Invocation:** RagService calls llmClient.generate(prompt). This call is delegated to the LLMClient interface, specifically implemented by llm.SiliconLLMClient. LLMClient is an adapter interface, making it easy to switch LLM providers in the future.

5. **Response Parsing:** RagService calls the private parseLLMResponse(rawResponse), using CITATION_PATTERN (a regular expression) and GENERAL_KNOWLEDGE_MARKER (a constant) to parse the raw String returned by the LLM. We do not return the String directly; instead, we encapsulate it in an Answer data object. This is a **robust, data-driven design** that structures the LLM's reply, clearly separating the "answer text," "citation list (List<Citation>)," and "whether it is general knowledge (isFallback flag)."

D. **Result Rendering (UI Rendering)** This path is responsible for safely rendering the backend processing results on the UI.

1. **Return to EDT:** The SwingWorker's done() method is triggered. When the background task (Path C) is complete, SwingWorker automatically and safely calls back the done() method on the UI thread (EDT). This avoids problems with multi-threaded concurrent modification of the UI.
2. **UI Rendering:** In the done() method, ChatPanel.appendMessage("TA: " + answer.getText(), ..., answer) is called.
3. **Conditional Rendering:** The appendMessage method internally checks answer.isFallback(), which is key to separating "RAG" and "general knowledge" in the UI. Based on the isFallback flag in the Answer object, we **dynamically decide** whether to call the renderCitationCards(citations) method to render citation cards, making the UI display logic driven entirely by the structured Answer object returned from the backend.

4.2.2 F2: AI Code Modification Workflow



F2 Workflow Diagram

The F2 (AI Code Modification) function is a pure Code-to-Code transformation process. Its design focuses on safety (using a Diff view and safe writes), streaming response (real-time display of the AI generation process), and iterative (Conversational) modification.

A. Trigger & Instruction Input

1. **Trigger:** The user selects code in the editor and right-clicks, triggering the actions.ModifyCodeWithAIAction class registered in plugin.xml.
2. **Context Gathering:** The ModifyCodeWithAIAction.actionPerformed() method is called. It retrieves CommonDataKeys.EDITOR from the AnActionEvent and calls editor.getSelectionModel().getSelectedText() to extract the "original code" String.
3. **Instruction Input:** actionPerformed() calls the private showInlineInputPanel(...) method. This method creates and displays a JBPopup (an IntelliJ-style floating window). We built a custom JBPopup containing a JComboBox<UIAIModel>

(model selection) and a JBTextField (instruction input). This allows the user to select the most suitable AI model (e.g., one skilled at code refactoring) while inputting the instruction, providing greater flexibility.

B. Asynchronous Backend Processing When the user clicks "Send" in the JBPopup, the showAsynchronousDiff(...) method is called, which immediately starts a background task.

1. **Start Background Task:** The showAsynchronousDiff() method calls ProgressManager.getInstance().run(new Task.Backgroundable(...)). This asynchronous processing method is used to execute time-consuming network requests. It displays a progress bar at the bottom of the IDE while **preventing the UI thread (EDT) from freezing**.

2. **Prompt Building:** In the background task's run() method, AIModificationUtil.buildCodeModificationPrompt(...) is called to assemble the "user instruction" and "original code" into a String Prompt specifically for code generation (containing strict rules, such as "return only code"). AIModificationUtil is a dedicated utility class that encapsulates the Prompt Engineering and LLM call logic for F2, keeping the responsibilities of ModifyCodeWithAIAction (the AnAction class) clean (focused only on UI and IntelliJ interaction).

3. **Streaming LLM Invocation:** The background task calls AIModificationUtil.modifyCodeStream(...), which in turn creates a SiliconLLMClient instance and calls its generateStream(...). This method is **streaming**; it returns AI-generated code snippets in real-time via a callback function (onChunk).

C. Diff Viewer & Streaming Display This is the core UI component of the F2 feature. It is created and displayed **before the background task is even started**.

1. **Diff Dialog Creation:** The showAsynchronousDiff() method creates a CodeModificationDialog instance. CodeModificationDialog inherits from DialogWrapper, the standard way to create modal dialogs in IntelliJ plugins. It internally encapsulates a DiffRequestPanel.

2. **Diff Panel Initialization:** The CodeModificationDialog's constructor calls DiffManager.getInstance().createRequestPanel() to create the native IntelliJ Diff view, providing a **native, high-performance, side-by-side code comparison experience with syntax highlighting**. The left side of the Diff view (originalContent) is bound to the "original code" string; the right side (modifiedContent) is bound to a newly created, editable Document object, with its initial content set to "Thinking...".

3. **Streaming Content Rendering:** When the onChunk callback of AIModificationUtil.modifyCodeStream in Path B is triggered, it calls dialog.appendTextChunk(chunk). The appendTextChunk method, under the protection of a WriteCommandAction, **appends** the String chunk returned by the AI to the Document object on the right panel. The user sees the AI generating code character by character (or chunk by chunk) in **real-time**, greatly enhancing the interactive experience.

D. Multi-turn Iterative Modification After Path C (the first streaming render) is complete, the F2 workflow does not end; it enters an iterative loop.

1. **Trigger:** The user inputs a new instruction (e.g., "Now add Javadoc for me") in the chatInput (a JBTextField) at the bottom of the CodeModificationDialog and clicks "Send." The CodeModificationDialog's setupChatListeners() captures this event and calls the onChatSubmit callback (which was registered by ModifyCodeWithAIAction when it was created in Path C).

2. **Context Passing:** The onChatSubmit callback triggers the ModifyCodeWithAIAction.continueModification(...) method.

3. **The Loop:** The continueModification method retrieves the current Document content from the right panel of the Diff view (i.e., the result of the previous AI generation). It uses this code as the new "original code," along with the user's new instruction, and once again starts a Task.Backgroundable, repeating Path B and Path C. The new code returned by the AI overwrites the content of the right panel. This design transforms F2 from a "one-shot" tool into a **conversational code refactorer**. It allows the user to make continuous, iterative changes based

on the first generation (e.g., "Great, now make it private") until they are completely satisfied with the result.

E. **Final Apply Changes**

1. **User Decision:** The user reviews the final Diff result after completing all iterations (Path D). If they click "Cancel," the DialogWrapper closes, and nothing happens. If they click "Apply" (OK), dialog.doOKAction() is triggered.
2. **Trigger Callback:** doOKAction() calls applyCallback.accept(finalCode), which is the final code retrieved from the modifiedContent Document on the right. The applyCallback is a Consumer<String>, defined in showAsynchronousDiff in Path A as (finalCode) -> applyChanges(project, editor, finalCode). This callback mechanism passes the "Apply Clicked" event from the Dialog class back to the Action class.
3. **Safe Write:** Technical Implementation: The applyChanges() method is called, which executes WriteCommandAction.runWriteCommandAction(...). Any modification to the editor's Document must be executed within a WriteCommandAction. This ensures: the operation is thread-safe, and the operation is registered in IntelliJ's Undo/Redo Stack. After applying the AI modification, the user can immediately press Ctrl+Z to undo the change.

4.3 RAG Core Data Structures and Algorithms

4.3.1 Document Chunking Strategy and Data Structure

To achieve efficient document retrieval, the system adopts customized chunking strategies for different document formats:

- **PDF and PPT Documents**: Naturally chunked by pages or slides, with each page/slide as an independent document chunk, preserving page-level metadata (page numbers/slide numbers) for precise citation of retrieval results.
- **Word Documents**: Adopt a sliding window chunking strategy with each chunk size of 800 characters, 100 characters overlap between adjacent chunks, and the window moving forward 700 characters each time, ensuring uniform chunk sizes for vectorization and retrieval while the overlap region prevents semantic loss across chunks.

- **Intelligent Boundary Splitting**: Word document chunking introduces an intelligent boundary splitting mechanism, cutting at punctuation marks in priority order: Period(。..) > Question/Exclamation marks(!?！？) > Semicolon(;；) > Line break(\n\n) > Space. The system backtracks up to 600 characters from the 800-character position to find appropriate punctuation; if none is found, it splits at the nearest space; if no boundary exists, it forces a split at the 800-character position. This strategy effectively avoids truncating complete sentences and maintains semantic integrity of document chunks.

All chunked document blocks use a unified PageText data structure:

```
public class PageText {
   public final String fileName;   // Source filename
   public final int page;          // PDF/PPT: page number; Word: chunk number(1,2,3...)
   public final String text;       // Text content of the document chunk
}
```

4.3.2 Index Structure

The system adopts a dual-index architecture to support keyword and semantic retrieval:

Lucene Inverted Index

Tokenizes and indexes the full text content of each document chunk (content field), while precisely storing the filename (source field) and page range (page field), with the index persisted to local disk.

Vector Index

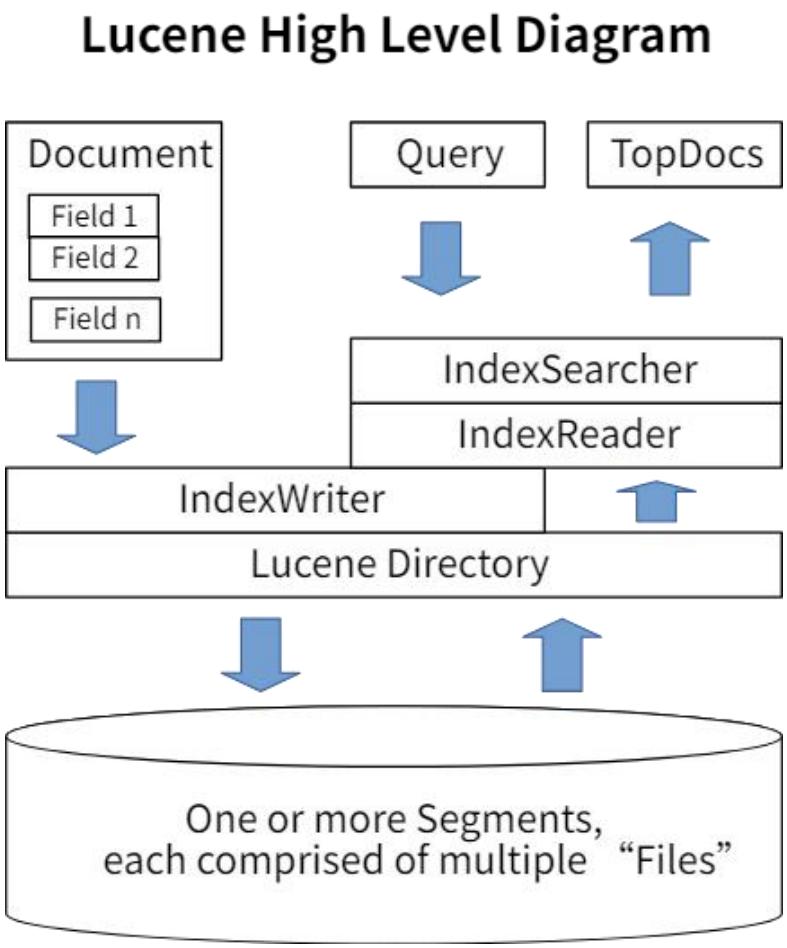
Uses fileName:pageNumber as the unique key (e.g., "Lecture3.pdf:5"), converts each document chunk into 384-dimensional or 1024-dimensional embedding vectors float[], stored in an in-memory HashMap to achieve O(1) lookup efficiency:

```
Map<String, float[]> index = new HashMap<>();
// Example: "Lecture3.pdf:5" -> [0.12, -0.34, ..., 0.56]
```

4.3.3 Retrieval Algorithms

The retrieval module of this system aims to balance semantic understanding and precise keyword matching, comprehensively adopting Lucene keyword retrieval, vector similarity retrieval, and hybrid retrieval algorithms. Lucene retrieval has extremely high precision and response speed when handling explicit technical terms or proper nouns; vector retrieval relies on embedding models with semantic recognition and fuzzy matching capabilities; hybrid retrieval fuses the results of both through weighted score fusion, balancing precision and semantic robustness, significantly improving recall and accuracy in complex query scenarios in actual tests.

Lucene Keyword Retrieval

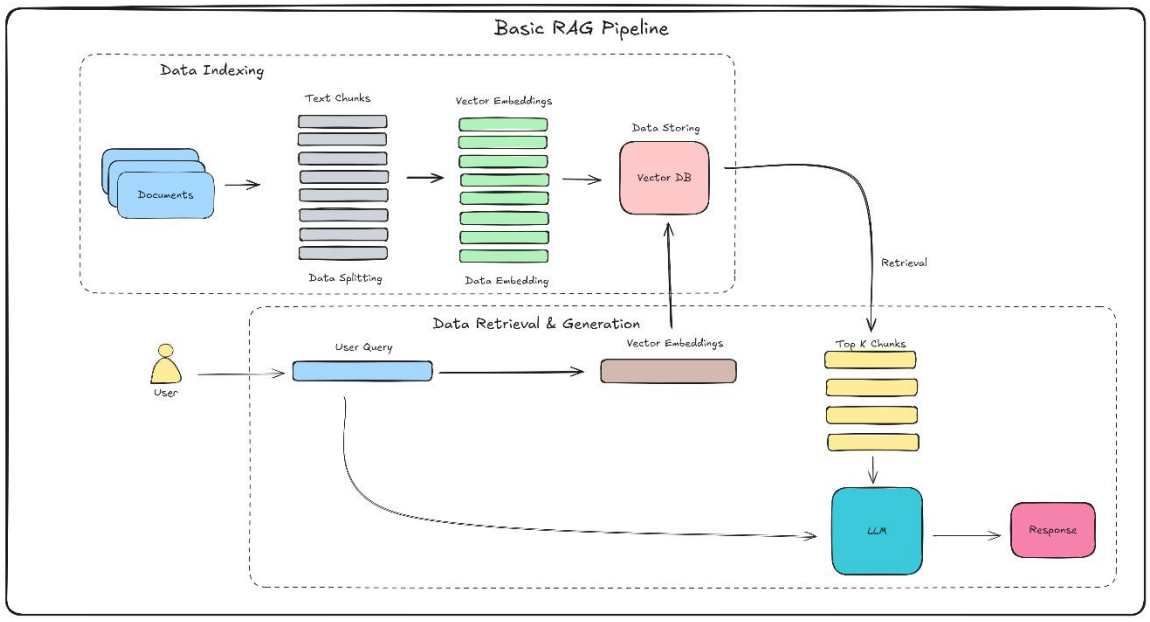


- **Similarity Algorithm**: The system adopts the BM25 (Best Match 25) algorithm, which comprehensively considers term frequency (TF) and inverse document frequency (IDF), while normalizing document length to avoid long documents dominating, particularly suitable for precise keyword matching scenarios (such as

searching for "Java polymorphism", "interface implementation" and other technical terms).

- **Query Processing Flow**:
    1. Use StandardAnalyzer to tokenize user queries
    2. Build Boolean queries (BooleanQuery) supporting multi-word combinations
    3. Match document chunks containing query terms in the inverted index
    4. Sort by BM25 score and return Top-K most relevant document chunks
- **Advantages**: Fast (millisecond-level response), accurate matching for proper nouns and technical terms, supports Boolean logic (AND/OR) and phrase queries.

Vector Similarity Retrieval



- **Similarity Metric**: Uses Cosine Similarity with the formula similarity = (A · B) / (||A|| × ||B||), measuring the consistency of vector directions independent of vector length, with value range in [-1, 1] (usually results in [0, 1]), where larger values indicate greater semantic similarity.

```
private double cosine(float[] a, float[] b) {

   double dot = 0.0, normA = 0.0, normB = 0.0;

   for (int i = 0; i < a.length; i++) {
```

```
        dot += a[i] * b[i];     // Dot product
        normA += a[i] * a[i];   // Square of A's magnitude
        normB += b[i] * b[i];   // Square of B's magnitude
    }
    return dot / (Math.sqrt(normA) * Math.sqrt(normB));
}
```

- **Retrieval Flow**:
    1. Query Vectorization: Call embedding model qVec = predictor.predict(query) to convert query text into 384-dimensional vectors
    2. Brute Force Traversal: Traverse all document vectors in the HashMap
    3. Similarity Calculation: Calculate cosine similarity between each document vector and the query vector
    4. Sort and Return: Sort in descending order of similarity and return Top-K most similar document chunks
- **Advantages and Disadvantages**:
    - Advantages: Understands semantics (e.g., "inheritance" and "subclassing" are semantically similar), supports fuzzy matching (spelling errors, synonyms), suitable for conceptual questions (e.g., "what is object-oriented")
    - Disadvantages: High computational cost (requires traversing all vectors), less precise than keyword retrieval for proper nouns

Hybrid Retrieval Algorithm

- **Fusion Strategy**: Adopts Weighted Score Fusion strategy, normalizing Lucene's BM25 scores to [0, 1], then fusing using the formula finalScore = α × luceneScore + (1-α) × vectorScore, with default α = 0.6 (Lucene weight 60%, vector weight 40%), adjustable based on actual retrieval performance (technical documents recommend α=0.6-0.7).

- **Deduplication Logic**: Uses filename:page/chunk number as the unique key; if both Lucene and vector retrieval return the same document chunk, they are merged into one record.

```
private String key(RetrievedChunk r) {
    return r.getSourceFile() + ":" + r.getPageRange();
    // Example: "Lecture3.pdf:p5" or "notes.docx:3"
}
```

- **Fusion Flow**:
    1. **Parallel Retrieval**: Simultaneously call Lucene and vector retrieval, each returning Top-K results
    2. **Result Merging**: Store using HashMap with key as fileName:page; when the same document chunk appears in both result sets, perform weighted fusion; when appearing in only one result set, retain original score
    3. **Weighted Fusion Calculation**: For example, document chunk "Lecture3.pdf:p5" has Lucene score 0.85, vector score 0.72, fusion score = 0.6 × 0.85 + 0.4 × 0.72 = 0.798
    4. **Re-ranking**: Sort in descending order by fused scores
    5. **Top-K Truncation**: Return K document chunks with highest scores (default K=5)

- **Hybrid Retrieval Advantages**: Complementarity (when keyword matching fails, vector retrieval may find semantically relevant content), robustness (when one retrieval method fails, another can supplement), precision improvement (experiments show hybrid retrieval improves accuracy by 15-25% over single methods).

4.3.4 Retrieval Result Data Structure

Retrieval results are uniformly encapsulated in the RetrievedChunk class:

```
public class RetrievedChunk {
    private final String sourceFile;   // Source filename (e.g., "Lecture3.pdf", "notes.docx")
    private final String pageRange;    // Page number/chunk number (e.g., "p5", "3")
    private final String content;      // Complete text content of the document chunk
```

```
    private final double score;       // Relevance score [0, 1], higher is more relevant



    public String getSourceFile() { return sourceFile; }

    public String getPageRange() { return pageRange; }

    public String getContent() { return content; }

    public double getScore() { return score; }

}
```

Field Descriptions:

- **sourceFile**: Used to display source in UI (e.g., "This answer is from Lecture3.pdf")
- **pageRange**: For PDF, represents page number (e.g., "p5" means page 5); for Word, represents chunk number (e.g., "3" means 3rd document chunk); for PPT, represents slide number (e.g., "p2" means 2nd slide)
- **content**: Complete document chunk text, used to pass to LLM as context or display original text summary in UI
- **score**: Used for sorting and confidence display (0.8-1.0: highly relevant, 0.6-0.8: moderately relevant, 0.4-0.6: weakly relevant, <0.4: basically irrelevant)

Typical Retrieval Result Example:

```
List<RetrievedChunk> results = [

    RetrievedChunk {

        sourceFile: "Lecture3.pdf",

        pageRange: "p12",

        content: "Java polymorphism means the same method call can have different behaviors...",

        score: 0.92

    },

    RetrievedChunk {

        sourceFile: "notes.docx",

        pageRange: "5",
```

```
    content: "Three necessary conditions for polymorphism: inheritance, overriding, parent class reference pointing to child class object...",
    score: 0.85
  }
]
```

Use Cases: Building RAG Prompt (concatenating Top-3 content as context), UI reference annotation (displaying "Source: Lecture3.pdf Page 12 (Relevance: 92%)"), result highlighting (annotating matched keywords in original text).

4.3.5 Fault Tolerance and Degradation Mechanism

- **Vector Index Degradation**: If local model loading fails (e.g., network issues or missing dependencies), the system automatically disables vector search functionality (isEnabled = false) and only uses Lucene keyword retrieval, ensuring basic system functionality remains available.
- **API Call Fault Tolerance**: When cloud API fails, returns empty vector retrieval results, and hybrid retrieval automatically degrades to pure Lucene retrieval.
- **Logging and User Feedback**: All exception situations have clear console log output and prompt users about the current retrieval mode being used.

4.3.6 Performance Optimization Considerations

- **Lucene Index Persistence**: Avoids repeated parsing and indexing of documents
- **Vector Index Memory Caching**: Uses HashMap to achieve O(1) lookup
- **Batch Embedding Optimization**: API version supports batch calls to reduce network overhead
- **Top-K Pruning**: Immediately prunes after score calculation to reduce subsequent processing overhead

4.4 LLM Integration and Prompt Engineering

4.4.1 LLM Service Integration

1. **LLM API Selection:** Our team chose **SiliconFlow** (api.siliconflow.cn) as the unified LLM API access point. The benefit of this is that SiliconFlow acts as an aggregator, allowing us to flexibly switch between different underlying models by simply changing the model parameter. Both the F0 (RAG Q&A) and F2 (Code Modification)

functions provide buttons to switch models, allowing users to choose between GLM, Deepseek, and Qwen models in the UI.

2. **API Connection Implementation:** Our connection implementation is located in the org.ta.llm.SiliconLLMClient class. It uses Java 11's built-in java.net.http.HttpClient to implement API connection and communication, keeping the plugin lightweight.

   - **JSON Data Structure:**

   Request Body: We use ObjectMapper (Jackson library) to serialize our SiliconFlowRequest POJO class into JSON. Its structure is as follows:

```
{
 "model": "deepseek-v2-chat",
 "messages": \[
  { "role": "user", "content": "..." }
 \],
 "stream": true // 用于 F2 的流式响应
}
```

   - **Response Body:**

     Non-streaming (F0): We deserialize the returned JSON response into a SiliconFlowResponse object and call getReplyContent() to extract the content of choices[0].message.content.

     Streaming (F2): In the generateStream method, we read the response line by line and parse the data: {...} format Server-Sent Events (SSE), deserializing them into StreamChunk objects to get the content block.

3. **Key Technologies:**

Asynchronous Execution: LLM calls are time-consuming network I/O operations. If executed in the UI thread (EDT) (like in Swing or IntelliJ UI), they will cause the entire IDE interface to freeze and hang. We used the correct asynchronous strategy for each of the two functions:

F0 (RAG Q&A): In ChatPanel.java, we used the standard Swing asynchronous tool SwingWorker<Answer, Void>. The network request (service.answer(...)) is executed in the doInBackground() background thread, and the result is safely returned to the UI thread via the done() method for rendering.

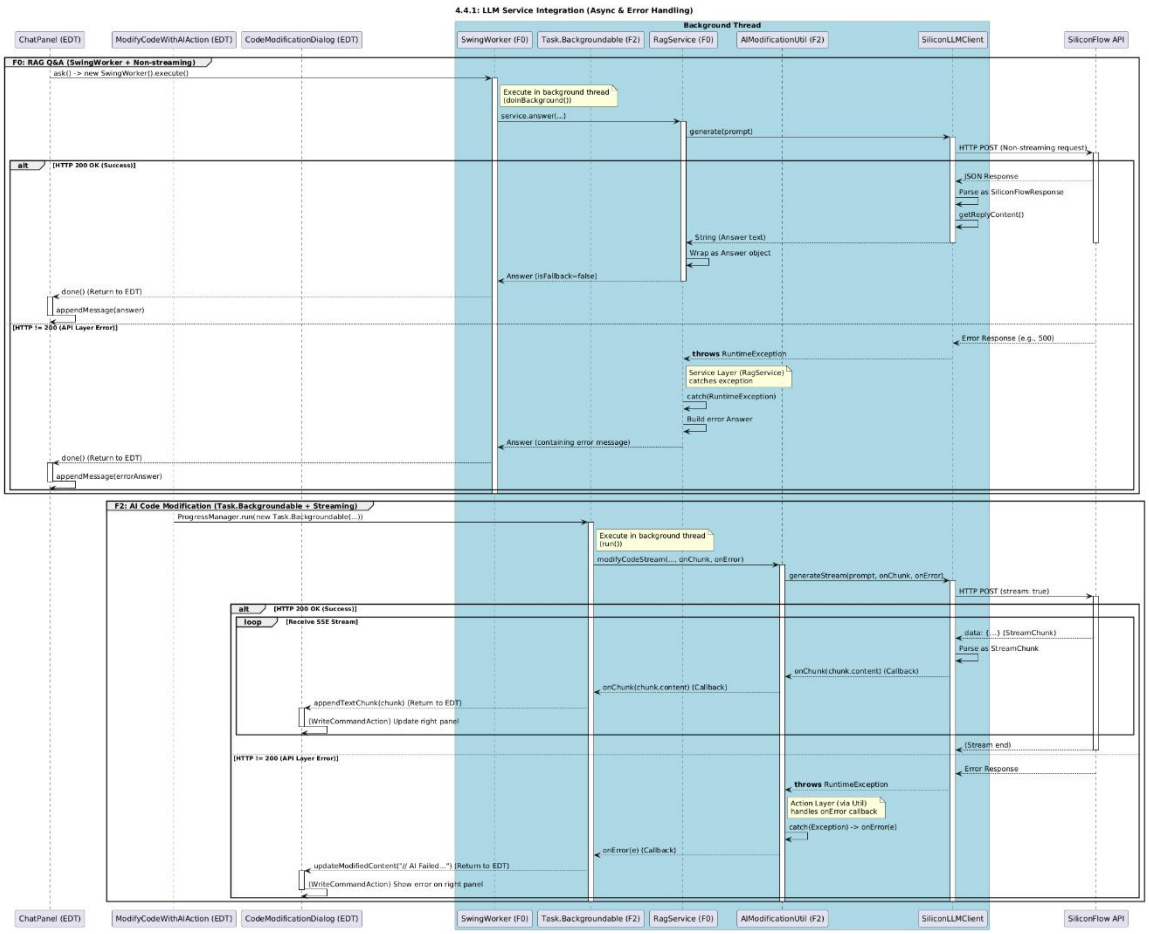
F2 (Code Modification): In ModifyCodeWithAIAction.java, we used the IntelliJ platform's **recommended** asynchronous tool: ProgressManager.getInstance().run(new Task.Backgroundable(...)). This not only places AIModificationUtil.modifyCodeStream(...) in a background thread but also displays a visible progress bar at the bottom of the IDE, providing a better user experience.

Error Handling: Our error handling is multi-layered:

API Layer (SiliconLLMClient): Checks the HTTP response status code. If it is not 200, it throws a RuntimeException.

Service Layer (F0 - RagService): Catches the RuntimeException and returns an Answer object containing an error message (e.g., "Sorry, an error occurred while calling the AI service."). The UI (ChatPanel) will display this error message as the AI's reply.

Action Layer (F2 - ModifyCodeWithAIAction): The modifyCodeStream method includes an onError callback. If an exception occurs, we call dialog.updateModifiedContent("// AI modification failed: " + e.getMessage()), safely rendering the error message as a comment on the right side of the Diff view, so the user can clearly see the reason for the failure.



LLM Asynchronous and Error Handling Workflow Diagram

4.4.2 Prompt Engineering

We designed two sets of highly structured prompts to ensure the LLM's output is **controllable, parsable, and safe**.

1. **F0 (RAG Q&A) Prompt:**
   - **Source:** org.ta.core.RagService.buildPrompt()
   - **Prompt (Java String):**

```
"You are a teaching assistant for the Java Enterprise Application Development course. Please strictly follow the \"[Background Knowledge]\" provided below to answer the \"[User Question]\".\n" +
"Rules:\n" +
"1. If the \"[Background Knowledge]\" is helpful for answering the question, your answer must incorporate information from it.\n" +
S "2. If you use information from the \"[Background Knowledge]\" you must clearly list all cited sources at the end of your answer in the format \"\".\n" +
```

```
        "3. If the \"[Background Knowledge]\" is completely irrelevant to the question or is empty, please use your own general knowledge to answer. However, your answer must start with the special marker \"[GENERAL_KNOWLEDGE]\" and must not contain any cite reference markers.\n" +
        "4. If the user provides \"[Code Context]\", please analyze and answer in conjunction with the code.\n" +
        "\n" +
        "\"[Background Knowledge]\"\n" +
        "---\n" +
        formattedChunks + "\n" +
        "---\n" +
        "\n" +
        "\"[Code Context]\"\n" +
        "---\n" +
        contextText + "\n" +
        "---\n" +
        "\n" +
        "\"[User Question]\"\n" +
        question
```

Design Rationale Analysis (Item by Item):

Role: You are a teaching assistant... -> **Defines the role and scope of knowledge**. This instructs the AI to answer in a tone appropriate for a TA and that its knowledge should primarily come from the course materials we provide.

Rule 1 (Citation): ...in the format "" -> **Enables machine-readable citations**. This is the core of F0. We force the AI to return citations in this strict format, allowing the RagService.parseLLMResponse() method to precisely parse these markers using CITATION_PATTERN (a regular expression) and render them as clickable citation cards in the ChatPanel.

Rule 2 (General Knowledge): ...start with the special marker "[GENERAL_KNOWLEDGE]" -> **Distinguishes RAG from general knowledge**. This is another core part of F0. If the RAG-retrieved materials (formattedChunks) are empty or irrelevant, the AI must begin its answer with this marker. Our parseLLMResponse() checks for this marker and sets the Answer object's isFallback flag accordingly. The UI layer (ChatPanel) uses this flag to decide whether to display a disclaimer ("(Answer based on general knowledge...)").

Markers: [Background Knowledge], "[Code Context]" and --- -> **Structures the input**. LLMs can easily get confused when processing long texts. We use these clear markers

and separators to delineate regions, clearly informing the LLM which part is the (Group A provided) knowledge, which part is the (user-selected) code, and which part is the user's question.

2. **F2 (Code Modification) Prompt:**

- **Source:** org.ta.core.AIModificationUtil.buildCodeModificationPrompt()
- **Full Prompt (Java String):**

```
"You are a senior Java architect and refactoring expert.\n" +
        "Your task is to modify the given Java code snippet according to the user's instructions.\n" +
        "Rules:\n" +
        "1. Strictly follow the user's instructions for modification.\n" +
        "2. Do not add any explanations, comments, or apologies unrelated to the code. Your response must be
purely, directly compilable Java code.\n" +
        "3. Maintain the indentation and formatting style of the original code.\n" +
        "4. If the instruction cannot be completed, return the original code.\n\n" +
        "\"[User Instruction]\"\n" +
        instruction + "\n\n" +
        "\"[Original Code]\"\n" +
        "```java\n" + code + "\n" + "```\n\n" +
        "\"[Modified Code]\"\n"
```

Design Rationale Analysis (Item by Item):

Role: You are a senior Java architect... -> **Switches the expert role for the task**. F2's task is not Q&A, but professional code refactoring. We switch the AI's role to "senior Java architect" to guide it toward generating higher-quality code that follows design patterns.

Rule 1 (Pure Code): ...must be purely, directly compilable Java code. -> **This is the most critical rule for F2**. The LLM's output will be **directly** fed into the Diff viewer of the CodeModificationDialog as the "modified code." If the AI adds any explanatory text (like Here is your modified code: or I have refactored the method:), this text will contaminate the code, cause the Diff to fail, and **break the user's source code** when they click "Apply." This rule forces the AI to output only code.

Auxiliary Measure (cleanCodeBlock): We added a cleanCodeBlock() helper method in both AIModificationUtil and TAService as a double-check, to programmatically strip these Markdown markers, ensuring that what is ultimately applied to the editor is pure code.

Rule 2 (Formatting): Maintain the indentation... -> **Preserves code style**. Prevents the AI from breaking or changing the user's original code indentation style during refactoring.

Markers: "[Original Code]" and ```java ... ``` -> **Helps the AI identify the code**. ```java is a Markdown marker that helps the LLM better enter "code mode" and recognize the code region, while the "[..." markers are used to clearly distinguish the user's instruction from the code to be modified.

4.5 Design Principles and Efficiency Considerations

Design Principles: We strictly adhere to IntelliJ's threading and interaction model: all potentially blocking operations (e.g., document retrieval, embedding computation, LLM requests) are executed as background tasks to avoid occupying the event dispatch thread, ensuring IDE fluency. UI updates follow event scheduling and appropriate modal states for thread safety and consistent UX. Code writing uses transactional write commands, supporting undo/redo and naturally integrating with the IDE's history mechanism. Streaming generation and incremental rendering are adopted for chat and code modification scenarios, giving continuous feedback during waiting; Diff review leverages native side-by-side views, with the right-side suggestions editable for continuous multi-turn iteration, ensuring coherent and controllable interactions.

Efficiency Considerations: Document indexing occurs once during service initialization, with subsequent queries performing only lightweight retrieval and formatting to maintain low latency in Q&A paths. Vector and keyword retrievals cooperate with weighted fusion for higher relevance and robustness. Network calls have reasonable timeouts and error handling; when necessary, fallback to local keyword search is supported, with progress and fallback messages maintaining continuity. Streaming transmission significantly reduces first response time, and returned blocks are cleaned to remove redundant Markdown wrappers and noise, minimizing unnecessary UI redraws. Overall resource management follows the principle of "create on demand, reuse long-lived objects"; LLM clients are instantiated lightly, indexes and caches are reused, and exceptions/logs are traced at key points for performance diagnosis and issue location.

5. Other Technical Details

The project is built on the IntelliJ platform, using Gradle and requiring JDK 17. Dependencies focus on retrieval and integration: PDF parsing uses Apache PDFBox; PPT/Word handled by Apache POI; keyword search via Apache Lucene 9.x; vector similarity via SiliconFlow Embeddings API; data encoding/decoding with Jackson; logging with SLF4J; UI with Swing components; basic rich text rendering via CommonMark.

For building and running, the org.jetbrains.intellij Gradle plugin is used, runIde launches in a sandbox; prepareSandbox packages runtime dependencies into the plugin's lib directory to avoid missing libraries. Network calls have reasonable timeouts and exception handling, falling back to local keyword search if needed. Code writing is unified via WriteCommandAction, ensuring safe integration with native Diff and undo mechanisms. Currently, the prototype manages models and API keys inline; the service layer supports model switching, which will be migrated to the settings page with persistence in the future. Chat and code sessions persist within window lifecycle and are cleared upon closing to avoid cross-file contamination.